Proiectarea și implementarea unui server DHCP concurent cu persistența alocărilor

Greavu Miruna-Ioana
Grupa C113D
Academia Tehnică Militară „Ferdinand I”
București, România

Busuioc Mario-Gabriel
Grupa C113D
Academia Tehnică Militară „Ferdinand I”
București, România

***Rezumat*—Lucrarea prezintă proiectarea și implementarea unui server DHCP care configurează automat stațiile dintr-o rețea locală. Serverul gestionează mai multe pool-uri de adrese și oferă trei moduri de alocare: statică, pe baza adresei fizice a clientului, dinamică, pe durata unui lease, și automată, permanentă. Înainte de a oferi o adresă, serverul verifică prin ARP și ICMP dacă aceasta este deja folosită în rețea. Lease-urile sunt urmărite pe tot ciclul lor de viață, de la alocare la reînnoire, eliberare și expirare, și sunt păstrate într-o bază de date, astfel încât starea serverului se reface după o repornire. Pe lângă adresa IP, clienții primesc masca de rețea, gateway-ul implicit, serverele DNS, numele de domeniu și alți parametri. Cererile sunt procesate în paralel de un grup de fire de execuție, sincronizate prin mutex-uri și semafoare, astfel încât aceeași adresă nu poate fi atribuită simultan la doi clienți. Funcționarea serverului este verificată într-o rețea virtuală izolată, alcătuită din două mașini virtuale.**

***Cuvinte-cheie*—DHCP, lease, pool de adrese, detecția conflictelor, fire de execuție, mutex, semafor, persistență**

I. Introducere

Orice stație dintr-o rețea IP are nevoie de o adresă unică, de masca subrețelei, de adresa gateway-ului implicit și de cea a serverelor DNS. Configurarea manuală a acestor parametri devine greu de întreținut pe măsură ce rețeaua crește și duce ușor la erori, cum ar fi două stații cu aceeași adresă. Protocolul DHCP (Dynamic Host Configuration Protocol), definit în RFC 2131 [1], rezolvă problema printr-un server central care distribuie automat configurația, iar opțiunile transmise clienților sunt standardizate în RFC 2132 [2].

Scopul proiectului este realizarea unui server DHCP funcțional, care să acopere mai mult decât schimbul de bază de mesaje. Serverul trebuie să gestioneze mai multe pool-uri de adrese, să ofere alocare statică, dinamică și automată și să verifice adresele înainte de a le distribui. De asemenea, trebuie să urmărească durata lease-urilor, să furnizeze parametri suplimentari de rețea, să stocheze lease-urile într-o bază de date și să trateze corect cererile concurente.

Lucrarea este organizată astfel. Secțiunea II prezintă noțiunile de bază ale protocolului. Secțiunea III enumeră cerințele funcționale, iar Secțiunea IV descrie arhitectura serverului. Secțiunea V detaliază modul în care este realizată fiecare cerință. Secțiunea VI descrie mediul și scenariile de testare, iar Secțiunea VII prezintă concluziile.

II. Fundamente teoretice

A. Formatul mesajelor DHCP

DHCP folosește protocolul UDP: serverul ascultă pe portul 67, iar clientul pe portul 68. Un mesaj DHCP are o parte fixă de 236 de octeți, moștenită de la protocolul BOOTP, urmată de valoarea *magic cookie* 99.130.83.99 și de o listă de opțiuni. Cele mai importante câmpuri ale părții fixe sunt:

- `op` — tipul mesajului: cerere (1) sau răspuns (2);
- `xid` — identificatorul tranzacției, ales de client, care leagă răspunsurile de cereri;
- `flags` — bitul de broadcast, prin care clientul cere ca răspunsul să fie trimis prin difuzare;
- `ciaddr` — adresa curentă a clientului, completată doar când clientul are deja o adresă validă;
- `yiaddr` — adresa oferită sau alocată clientului de către server;
- `giaddr` — adresa agentului releu, nenulă când mesajul a trecut printr-un router;
- `chaddr` — adresa hardware (MAC) a clientului.

Opțiunile sunt codificate în format TLV (*Type–Length–Value*): un octet pentru cod, un octet pentru lungime și valoarea propriu-zisă. Lista se încheie cu opțiunea 255 (*End*). Tipul mesajului DHCP este transmis prin opțiunea 53.

B. Tipurile de mesaje și ciclul DORA

Un client nou obține configurația printr-un schimb de patru mesaje, numit ciclul DORA:

1) **DHCPDISCOVER** — clientul, care nu are încă o adresă, caută servere DHCP printr-un mesaj de difuzare;
2) **DHCPOFFER** — serverul propune o adresă și parametrii de rețea;
3) **DHCPREQUEST** — clientul acceptă una dintre oferte și indică serverul ales prin opțiunea 54 (*Server Identifier*);
4) **DHCPACK** — serverul confirmă alocarea, iar din acest moment lease-ul este activ.

În afara acestor patru mesaje, protocolul mai definește: DHCPNAK, prin care serverul refuză o cerere invalidă; DHCPDECLINE, prin care clientul anunță că adresa primită este deja folosită; DHCPRELEASE, prin care clientul eliberează adresa; și DHCPINFORM, prin care un client cu adresă configurată manual cere doar ceilalți parametri de rețea.

C. Lease-ul și timerele asociate

Un lease este dreptul unui client de a folosi o adresă pentru o perioadă limitată, numită durata lease-ului (L). Pentru a nu pierde adresa, clientul încearcă să o reînnoiască la două momente:

$$T_1 = 0{,}5 \cdot L, \qquad T_2 = 0{,}875 \cdot L. \tag{1}$$

La momentul T_1 clientul trimite serverului care i-a alocat adresa o cerere de reînnoire, direct (*unicast*). Dacă nu primește răspuns până la T_2, trimite cererea prin difuzare, pentru a fi preluată de orice server disponibil. Dacă nici atunci nu reușește, la momentul L adresa expiră, iar clientul trebuie să reia ciclul DORA.

III. Cerințe funcționale

Tabelul I prezintă cerințele funcționale ale serverului și secțiunea în care este descrisă realizarea fiecăreia.

TABELUL I
CERINȚELE FUNCȚIONALE ALE SERVERULUI

Cod	Cerință	Secțiune
RF1	Gestiunea mai multor pool-uri de adrese	V-A
RF2	Alocare statică, dinamică și automată	V-B
RF3	Verificarea adreselor prin ARP și ping	V-C
RF4	Durata lease-ului, reînnoirea, eliberarea și expirarea	V-D
RF5	Furnizarea de parametri suplimentari (gateway, DNS, domeniu etc.)	V-E
RF6	Stocarea lease-urilor într-o bază de date	V-F
RF7	Tratarea concurenței dintre firele de execuție, cu mutex-uri și semafoare	V-G

IV. Arhitectura serverului

Serverul este scris în limbajul C, pentru sistemul de operare Linux, și folosește biblioteca POSIX Threads pentru firele de execuție, mutex-uri și semafoare. Aplicația este împărțită în module cu responsabilități separate:

- **Modulul de configurare** citește la pornire un fișier text cu pool-urile, rezervările statice, parametrii de rețea și durata implicită a lease-urilor.
- **Receptorul** ascultă pe portul UDP 67, cu opțiunea `SO_BROADCAST` activată, și pune mesajele primite într-o coadă de așteptare.
- **Grupul de fire de lucru** (*thread pool*) preia mesajele din coadă și le procesează în paralel.
- **Parserul și serializatorul de pachete** transformă octeții primiți în structuri de date și construiesc răspunsurile, inclusiv lista de opțiuni TLV.
- **Managerul de pool-uri și lease-uri** păstrează în memorie starea fiecărei adrese și aplică regulile de alocare.
- **Detectorul de conflicte** verifică, prin ICMP și ARP, dacă o adresă este deja folosită în rețea.
- **Modulul de persistență** salvează lease-urile în baza de date și le reîncarcă la pornire.
- **Firul de curățare** (*reaper*) eliberează periodic lease-urile și ofertele expirate.
- **Modulul de jurnalizare** înregistrează evenimentele: tipul mesajului, adresa MAC, adresa IP și momentul de timp.

La sosirea unui mesaj, receptorul îl adaugă în coadă, iar un fir de lucru liber îl preia, îl decodifică și determină tipul lui din opțiunea 53. În funcție de tip, firul apelează funcția de tratare corespunzătoare, care modifică starea lease-urilor și construiește răspunsul. Firul trimite apoi răspunsul și salvează modificările în baza de date.

V. Implementarea funcționalităților

A. Gestiunea pool-urilor de adrese

Un pool reprezintă un interval continuu de adrese dintr-o subrețea, împreună cu parametrii care se transmit clienților din acea subrețea. Fiecare pool are un identificator, adresa subrețelei și masca ei, adresa de început și de sfârșit a intervalului, gateway-ul, serverele DNS, numele de domeniu și durata implicită a lease-ului. Adresa serverului, a gateway-ului și eventualele adrese excluse explicit nu sunt distribuite niciodată.

Când clientul se află în aceeași rețea cu serverul, se folosește pool-ul care conține adresa interfeței pe care a sosit cererea. Când cererea a trecut printr-un agent releu, câmpul `giaddr` este nenul și se alege pool-ul care conține această adresă. Astfel, același server poate deservi mai multe subrețele.

În memorie, fiecare adresă dintr-un pool are una dintre stările din Tabelul II.

TABELUL II
STĂRILE UNEI ADRESE DIN POOL

Stare	Semnificație
LIBERĂ	Poate fi oferită oricărui client.
OFERITĂ	A fost propusă printr-un DHCPOFFER și este blocată câteva secunde, în așteptarea răspunsului clientului.
ALOCATĂ	Este folosită de un client, pe baza unui lease activ.
REZERVATĂ	Este rezervată static pentru o anumită adresă MAC.
INDISPONIBILĂ	A fost găsită ocupată în rețea sau refuzată printr-un DHCPDECLINE și este ținută în carantină.

Configurația pool-urilor se citește dintr-un fișier text. Un exemplu, pentru rețeaua folosită la testare:

```
[pool lan]
subnet      = 192.168.100.0/24
range       = 192.168.100.100 - 192.168.100.200
router      = 192.168.100.1
dns         = 192.168.100.1, 8.8.8.8
domain      = lab.local
lease_time  = 3600

[reservation]
mac = 00:50:56:26:79:f5
ip  = 192.168.100.50
```

B. Tipurile de alocare

Serverul oferă cele trei mecanisme de alocare definite în RFC 2131 [1]:

- **Alocarea statică** (rezervarea). Administratorul leagă o adresă IP de adresa MAC a unui client, iar clientul primește întotdeauna aceeași adresă. Mecanismul este util pentru imprimante, servere sau echipamente care trebuie să fie mereu accesibile la aceeași adresă. Adresele rezervate nu sunt oferite altor clienți.
- **Alocarea dinamică**. Clientul primește o adresă liberă din pool pentru o durată limitată L. Dacă nu reînnoiește lease-ul, adresa revine în pool și poate fi oferită altui client. Acesta este modul implicit de funcționare.
- **Alocarea automată**. Clientul primește o adresă liberă din pool, dar permanent. Serverul transmite în opțiunea 51 valoarea `0xFFFFFFFF`, care înseamnă durată infinită, iar adresa nu expiră și nu este reciclată de firul de curățare.

Clientul este identificat prin opțiunea 61 (*Client Identifier*), dacă aceasta există, altfel prin adresa MAC din câmpul `chaddr`. La primirea unui DHCPDISCOVER, adresa este aleasă în ordinea de preferință recomandată de standard: rezervarea statică a clientului, adresa pe care clientul o are deja sau a avut-o anterior, adresa cerută prin opțiunea 50 (*Requested IP Address*), dacă este liberă și validă, iar în ultimul rând prima adresă liberă din pool. Căutarea adresei și marcarea ei ca OFERITĂ se fac într-o singură secțiune critică, sub mutex-ul tabelei de lease-uri. După eliberarea mutex-ului, adresa este verificată în rețea (Secțiunea V-C): dacă este găsită ocupată, devine INDISPONIBILĂ și căutarea se reia, altfel serverul trimite DHCPOFFER. Dacă pool-ul nu mai are adrese libere, serverul nu răspunde cererii.

C. Verificarea adreselor prin ARP și ping

O adresă din pool poate fi totuși ocupată, de exemplu de o stație configurată manual. RFC 2131 [1] recomandă ca serverul să verifice adresa înainte de a o oferi, iar serverul face acest lucru în două moduri:

- **ICMP Echo (ping)**: serverul trimite o cerere *Echo Request* către adresa candidată și așteaptă un răspuns un timp scurt (de exemplu 500 ms). Orice răspuns înseamnă că adresa este folosită.
- **ARP**: când clientul se află în aceeași rețea cu serverul, acesta trimite o cerere ARP pentru adresa candidată. Metoda este mai sigură decât ping-ul, deoarece o stație poate bloca mesajele ICMP printr-un firewall, dar nu poate ignora cererile ARP fără să-și piardă conectivitatea.

Dacă se găsește un conflict, adresa este marcată INDISPONIBILĂ pentru o perioadă de carantină configurabilă, evenimentul este jurnalizat, iar serverul alege altă adresă. Același tratament se aplică atunci când un client trimite DHCPDECLINE, semn că el însuși a detectat un conflict după ce a primit adresa.

Verificarea durează până la câteva sute de milisecunde, deci nu se face cât timp firul ține blocat mutex-ul tabelei de lease-uri. Adresa este mai întâi marcată OFERITĂ, ceea ce o face invizibilă pentru celelalte fire, apoi mutex-ul este eliberat și abia după aceea se face verificarea. Astfel, verificarea unei adrese nu blochează procesarea cererilor celorlalți clienți.

D. Durata lease-ului, reînnoirea și eliberarea

La confirmarea unei alocări (DHCPACK), serverul salvează momentul de început t_0 și calculează momentul expirării:

$$t_{\text{exp}} = t_0 + L. \tag{2}$$

În răspuns sunt incluse opțiunile 51 (durata L), 58 (T_1) și 59 (T_2), calculate conform (1).

Mesajele DHCPREQUEST apar în patru situații diferite, pe care serverul le deosebește după conținutul mesajului:

- **Selecție** (din ciclul DORA): mesajul conține opțiunile 54 și 50. Dacă identificatorul din opțiunea 54 este al serverului, acesta confirmă adresa oferită; altfel, clientul a ales alt server, iar oferta este anulată.
- **Repornirea clientului**: mesajul conține opțiunea 50, dar nu și opțiunea 54. Serverul răspunde cu DHCPACK dacă adresa cerută aparține clientului și este validă în subrețea, altfel cu DHCPNAK.
- **Reînnoire** (la T_1) și **realocare** (la T_2): câmpul `ciaddr` este completat. Dacă lease-ul există, serverul îl prelungește, recalculând t_{exp} conform (2).

La primirea unui DHCPRELEASE, adresa redevine LIBERĂ imediat. Serverul păstrează totuși asocierea dintre client și adresă, astfel încât, dacă adresa nu a fost ocupată între timp, același client o primește din nou la următoarea cerere. Clienții care nu trimit DHCPRELEASE, de exemplu pentru că au fost opriți brusc, sunt tratați de firul de curățare: acesta rulează periodic, la câteva secunde, readuce în starea LIBERĂ toate adresele cu t_{exp} depășit și anulează ofertele la care clientul nu a răspuns în timpul alocat. Lease-urile automate și rezervările statice nu sunt atinse de firul de curățare.

E. Furnizarea parametrilor suplimentari

Pe lângă adresa IP, serverul transmite parametrii din Tabelul III, preluați din configurația pool-ului [2]. Opțiunile obligatorii sunt incluse în fiecare DHCPOFFER și DHCPACK, iar celelalte sunt trimise dacă apar în lista de parametri ceruți de client (opțiunea 55).

TABELUL III
OPȚIUNILE DHCP FURNIZATE DE SERVER

Cod	Opțiune	Exemplu
1	Masca subrețelei	255.255.255.0
3	Gateway implicit	192.168.100.1
6	Servere DNS	192.168.100.1, 8.8.8.8
15	Numele de domeniu	lab.local
28	Adresa de broadcast	192.168.100.255
42	Servere NTP	192.168.100.1
51	Durata lease-ului	3600 s
53	Tipul mesajului	OFFER, ACK, NAK
54	Identificatorul serverului	192.168.100.1
58	Timpul de reînnoire T_1	1800 s
59	Timpul de realocare T_2	3150 s

Același mecanism este folosit pentru DHCPINFORM: clientul are deja o adresă configurată manual, așa că serverul răspunde cu un DHCPACK care conține doar opțiunile cerute, fără să aloce o adresă și fără să creeze un lease.

F. Stocarea lease-urilor în baza de date

Pentru ca serverul să nu uite alocările făcute atunci când este oprit sau repornit, lease-urile sunt salvate într-o bază de date SQLite. SQLite a fost ales pentru că nu necesită un server de baze de date separat: întreaga bază este un singur fișier pe disc, accesat printr-o bibliotecă inclusă direct în aplicație. Structura tabelei este:

```
CREATE TABLE leases (
  ip          TEXT PRIMARY KEY,
  client_id   TEXT NOT NULL,     -- MAC / opt. 61
  hostname    TEXT,              -- opt. 12
  pool_id     TEXT NOT NULL,
  type        INTEGER NOT NULL, -- 0 dinamic
                                -- 1 static
                                -- 2 automat
  state       INTEGER NOT NULL,
  start_time  INTEGER NOT NULL, -- timp Unix
  expire_time INTEGER           -- NULL: permanent
);
CREATE INDEX idx_client ON leases(client_id);
```

Fiecare modificare a unui lease (alocare, reînnoire, eliberare, expirare, marcare ca indisponibilă) este scrisă în baza de date printr-o tranzacție. Tabela din memorie rămâne sursa folosită pentru deciziile de alocare, deoarece este mult mai rapidă, iar baza de date asigură persistența. La pornire, serverul încarcă lease-urile din baza de date, le elimină pe cele expirate între timp și reconstruiește starea fiecărei adrese din pool. Cele două structuri sunt astfel mereu sincronizate.

G. Tratarea concurenței

Mai mulți clienți pot trimite cereri în același timp, de exemplu când pornesc simultan toate calculatoarele dintr-un laborator. Serverul procesează aceste cereri în paralel, folosind modelul producător–consumator [3]:

- **Producătorul** este firul receptor, care citește mesajele de pe socket și le pune într-o coadă circulară de capacitate fixă N.
- **Consumatorii** sunt K fire de lucru, create la pornire, care preiau mesajele din coadă și le procesează.

Coada este sincronizată cu două semafoare cu numărare și un mutex. Semaforul *sem_libere*, inițializat cu N, numără pozițiile libere din coadă și blochează producătorul când coada este plină. Semaforul *sem_ocupate*, inițializat cu 0, numără mesajele care așteaptă procesarea și blochează consumatorii când coada este goală. Mutex-ul *mutex_coada* asigură că doar un fir modifică indicii cozii la un moment dat. Fiecare fir de lucru rulează într-o buclă: așteaptă la *sem_ocupate* până când în coadă există un mesaj, îl extrage sub protecția lui *mutex_coada* și semnalează *sem_libere*, pentru ca producătorul să știe că s-a eliberat o poziție. Mesajul este procesat abia după eliberarea mutex-ului, astfel încât celelalte fire pot folosi coada în paralel.

Resursa cea mai importantă este tabela de lease-uri, protejată de *mutex_lease*. Fără acest mutex, două fire care procesează simultan cererile a doi clienți ar putea găsi aceeași adresă liberă și ar putea-o oferi amândurora. Căutarea unei adrese libere și marcarea ei ca OFERITĂ se fac de aceea într-o singură secțiune critică (Secțiunea V-B). Secțiunile critice sunt păstrate cât mai scurte, iar operațiile lente, precum verificarea adresei prin ARP sau ICMP și trimiterea răspunsului, se fac în afara lor.

Baza de date are propriul mutex, *mutex_bd*. Pentru a evita blocajele reciproce (*deadlock*), mutex-urile sunt obținute mereu în aceeași ordine: *mutex_lease*, apoi *mutex_bd*, niciodată invers.

Planificarea firelor este împărțită între sistemul de operare și aplicație. Planificatorul nucleului Linux distribuie timpul de procesor între firele aplicației. Aplicația stabilește în ce ordine sunt tratate cererile: coada FIFO garantează că mesajele sunt procesate în ordinea sosirii, iar numărul fix de fire de lucru limitează consumul de resurse la o încărcare mare. Firul de curățare rulează independent, la intervale regulate, și folosește același *mutex_lease* când modifică starea adreselor. Prin urmare, expirarea unui lease nu poate interfera cu o alocare aflată în desfășurare.

VI. Testare și validare

Testarea se face într-o rețea virtuală izolată, creată în VMware Workstation. Rețeaua conține două mașini virtuale cu Ubuntu 26.04 LTS, conectate la același segment LAN (*LAN segment*), unde nu există niciun alt server DHCP:

- **dhcp-server**, cu adresa statică 192.168.100.1/24, pe care rulează serverul;
- **dhcp-client**, fără adresă configurată, care cere configurația cu utilitarul `dhclient`.

Izolarea este obligatorie: un server DHCP pornit într-o rețea reală, unde există deja un server DHCP, ar putea distribui adrese greșite celorlalte stații. Schimbul de mesaje este urmărit pe server cu `tcpdump` și Wireshark, iar pe client cu `dhclient -v`, care afișează fiecare pas al ciclului DORA. Tabelul IV prezintă scenariile de testare și comportamentul așteptat.

Scenariul 9 se poate realiza cu un client simulat, care trimite rapid mai multe mesaje DHCPDISCOVER cu adrese MAC diferite. La final se verifică în baza de date că nu există două lease-uri active pentru aceeași adresă IP.

VII. Concluzii

Lucrarea a prezentat proiectarea unui server DHCP care acoperă ciclul de viață complet al unei adrese: alegerea ei dintr-un pool, verificarea în rețea, alocarea statică, dinamică sau automată, reînnoirea, eliberarea și expirarea. Persistența în baza de date permite serverului să își refacă starea după o repornire. Procesarea concurentă, cu un grup de fire de lucru sincronizate prin semafoare și mutex-uri, permite tratarea simultană a mai multor clienți fără ca aceeași adresă să fie oferită de două ori.

Serverul poate fi extins în mai multe direcții: suport complet pentru agenți releu și pentru opțiunea 82, funcționarea a două servere în configurație redundantă (*failover*), o interfață de administrare pentru vizualizarea lease-urilor și a gradului de ocupare a pool-urilor și suport pentru DHCPv6, în rețelele IPv6.

TABELUL IV
SCENARIILE DE TESTARE

Nr.	Scenariu	Rezultat așteptat
1	Client nou: `sudo dhclient -v ens33`	Ciclul DORA complet; clientul primește o adresă din intervalul .100–.200, masca, gateway-ul, serverele DNS și domeniul.
2	Client cu rezervare statică	Clientul primește întotdeauna adresa rezervată, indiferent de ordinea cererilor.
3	Adresă ocupată de o stație configurată manual	Verificarea ARP sau ping detectează conflictul; adresa devine INDISPONIBILĂ, iar clientul primește altă adresă.
4	Reînnoirea lease-ului (durată scurtă, de exemplu 60 s)	La T_1 clientul trimite DHCPREQUEST, iar serverul prelungește lease-ul și actualizează baza de date.
5	Eliberarea adresei: `sudo dhclient -r ens33`	Serverul primește DHCPRELEASE și marchează imediat adresa ca LIBERĂ.
6	Expirarea lease-ului (clientul este oprit brusc)	După t_{exp}, firul de curățare readuce adresa în starea LIBERĂ.
7	Cerere pentru o adresă din altă subrețea	Serverul răspunde cu DHCPNAK, iar clientul reia ciclul DORA.
8	Repornirea serverului	Lease-urile active sunt reîncărcate din baza de date, iar clienții își păstrează adresele.
9	Cereri simultane de la mai mulți clienți simulați	Fiecare client primește o adresă diferită; nicio adresă nu este atribuită de două ori.

BIBLIOGRAFIE

[1] R. Droms, "Dynamic Host Configuration Protocol," RFC 2131, IETF, March 1997.

[2] S. Alexander and R. Droms, "DHCP options and BOOTP vendor extensions," RFC 2132, IETF, March 1997.

[3] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA: Pearson, 2015.